Modelación de sistemas multiagentes con gráficas computacionales (Gpo 570)

Evidencia 1.

David Isaí Díaz Avilés (A01666448)

Edgar Martínez Retes (A01706825)

Elisheba Hannai Trejo Leyva (A01736967)

Ian García González (A01706892)

Óscar Orlando De la Paz Ibarra (A01644484)

Pablo Javier Arreola Velasco (A01747824)

2 de Marzo de 2026

Descripción del reto a desarrollar

El reto consiste en desarrollar una simulación de una flota de robots autónomos que operan dentro de un almacén inicialmente desordenado. En el entorno se generan cajas ubicadas aleatoriamente sobre el piso, las cuales deben ser recogidas y organizadas en pilas de hasta cinco unidades.

Cada robot actúa como un agente autónomo dentro de un sistema multiagente, con capacidades de percepción, decisión y acción. El sistema debe permitir que los robots planifiquen y ejecuten sus movimientos de manera eficiente, evitando obstáculos como paredes, otros robots o pilas ya formadas. Asimismo, se debe implementar un mecanismo básico de coordinación o reglas de interacción que reduzcan conflictos y colisiones.

El objetivo principal es optimizar la organización del almacén, minimizando tanto el tiempo total requerido para completar el apilamiento como el número de movimientos realizados por los robots. La simulación representa un modelo simplificado de coordinación y planificación distribuida en entornos logísticos automatizados, similar a los sistemas utilizados en almacenes inteligentes.

Identificación de agentes involucrados

Para la simulación elegimos una arquitectura simple con dos agentes

1. Agente Warehouse Manager

El Agente Warehouse Manager es el agente encargado de la supervisión general del almacén. Su rol dentro del sistema es coordinar el proceso de organización, monitoreando el estado de las cajas, las pilas formadas y la disponibilidad de los robots.

Este agente mantiene una visión global del entorno y actúa como punto de referencia para la asignación de tareas. No realiza acciones físicas dentro del almacén, sino que cumple una función de gestión y coordinación.

2. Agente Robot Autónomo

Los Agentes Robot Autónomo son los agentes encargados de la ejecución física de las tareas de organización. Cada robot puede desplazarse en las cuatro direcciones, recoger cajas en celdas adyacentes y transportarlas para formar pilas de hasta cinco cajas.

Cada robot interactúa directamente con el entorno, detectando obstáculos como paredes, otras pilas o robots, y ajustando su comportamiento para evitar colisiones. Además, mantienen comunicación con el Warehouse Manager para recibir tareas y reportar su estado.

Arquitectura Multiagente

El sistema propuesto se diseñó bajo un enfoque multiagente jerárquico, en el cual se diferencian niveles de toma de decisiones estratégica y operativa. Esta separación permite modelar distintos grados de inteligencia dentro del entorno, asignando responsabilidades específicas a cada tipo de agente y garantizando una coordinación eficiente en la organización del almacén.

Arquitectura del Agente Warehouse Manager

El Agente Warehouse Manager implementa una arquitectura deliberativa basada en el modelo BDI (Beliefs, Desires, Intentions). Esta elección responde a la necesidad de realizar planificación estratégica a nivel global, ya que este agente posee una visión completa del estado del entorno.

Dentro del modelo BDI:

- Creencias (Beliefs): Representan la información actualizada del sistema, incluyendo la ubicación de todas las cajas, el estado de las pilas, la posición y disponibilidad de los robots, así como el progreso general del almacén.
- Deseos (Desires): Corresponden al objetivo principal del sistema, que consiste en organizar todas las cajas en pilas de hasta cinco unidades minimizando tiempo y movimientos.

- Intenciones (Intentions): Son los planes concretos que el agente decide ejecutar, tales como la asignación específica de tareas a robots determinados, priorización de zonas o redistribución de cargas de trabajo.

Esta arquitectura deliberativa permite evaluar múltiples variables antes de tomar decisiones, optimizando la eficiencia global y reduciendo comportamientos redundantes. Dado que el Warehouse Manager no realiza acciones físicas directas, sino que coordina y planifica, el enfoque BDI resulta adecuado para representar razonamiento estratégico estructurado.

Arquitectura de los Agentes Robot Autónomo

Los Agentes Robot Autónomo implementan una arquitectura híbrida que combina componentes reactivos y deliberativos. Esta decisión se fundamenta en la naturaleza dinámica del entorno, donde es necesario equilibrar planificación con respuesta inmediata ante eventos imprevistos.

Componente Reactivo

El componente reactivo permite al robot:

- Detectar obstáculos como paredes, otras pilas o robots.
- Prevenir colisiones mediante ajustes inmediatos en su trayectoria.
- Responder a cambios inesperados en el entorno.

Este nivel garantiza seguridad operativa y adaptabilidad en tiempo real, aspectos fundamentales en sistemas físicos distribuidos.

Componente Deliberativo

El componente deliberativo permite al robot:

- Ejecutar la ruta asignada y gestionar su estado interno
- Decidir cómo apilar correctamente una caja sin exceder el límite permitido.
- Gestionar su estado interno (si transporta una caja o está disponible).

Aunque la asignación de tareas es centralizada, la ejecución local requiere cierto grado de planificación autónoma para evitar ineficiencias o bloqueos.

Integración de Niveles de Inteligencia

La combinación de una arquitectura deliberativa centralizada (Warehouse Manager) con arquitecturas híbridas distribuidas (Robots Autónomos) permite establecer un equilibrio entre:

- Optimización global del sistema.
- Autonomía local en la ejecución.
- Reactividad ante eventos dinámicos.

Este diseño reduce la complejidad de coordinación directa entre robots, evita conflictos frecuentes y mejora la eficiencia colectiva del sistema. Además, refleja un modelo simplificado de arquitecturas empleadas en entornos logísticos automatizados reales, donde existe planificación estratégica central junto con ejecución autónoma en cada unidad operativa.

Componentes Arquitectónicos de Cada Agente

Diagrama AUML

AUML AGENTE ROBOT

<agent>
AgenteRobot

Atributos

- id: int
- pos: (x, y)
- carrying: bool
- currentTask: Task(PickUpCrate | DropToStack | Park)
- assignedCrateId: int
- targetCell: (x, y)
- targetStackCell: (x, y)(drop zone)
- path: Queue<Direction>
- blockedTicks: int
- priority: int
- status: RobotStatus {Idle, ToPickup, ToDrop, Parked, Blocked}

Sensores

- senseAdjacent(): List<CellInfo>
- senseCarrying(): bool
- senseNextCellFree(nextDir): bool
- senseCollisionRisk(nextDir): bool
- senseAtTarget(): bool
- senseStackHeight(cell): int

Acciones

- requestMove(): MoveIntent
- commitMove(newCell)
- pickUp(adjCell)
- drop(stackCell)
- wait()
- clearPath()
- notifyManager(event)

Métodos

- onTaskAssigned(task, path)
- step()
- updateBlocked(isBlocked)

Roles

- /Contractor
- /Transporter

Protocolos

- AIP_TaskAssignment_Warehouse
- AIP_MotionArbitration

AUML AGENTE MANAGER

<agent>
WarehouseManager

Atributos

- id: string
- grid: Room [generado a base de k]
- width: int
- height: int
- robots: Map<robotId, RobotInfo>
- crates: Map<crateId, CrateInfo>
- tasksQueue: Queue<Task>
- assignments: Map<taskId, robotId>
- blockedRequests: Queue<MoveRequest>
- rngSeed: int
- metrics: { totalMoves:int, timeToComplete:int}
- maxStackHeight: int

Sensores

- observeWorld(): WorldSnapshot
- receiveMoveRequest(robotId, fromCell, toCell): bool
- reveiceEvent(robotId, event)
- getStackHeight(cell): int

Acciones

- publishTask (task)
- assignTask(robotId, task)
- sendMovePermission(robotId, toCell)
- sendMoveReject(robotId, reason)
- broadcastStateUpdate()
- notifyAll(message)

Métodos

- initWorld(width,height, crates, robots)
- generateTasksFromCrates()
- evaluateBids(task) -> bestRobot
- calculatePath(from, to) -> List<Direction>
- reservePath(robotId, path)
- arbitrateMoveRequest(robotId, from, to) -> (allow/deny)
- reassignIfStuck(robotId)
- logAndExportMetrics()

Roles

- /Manager
- /TaskCoordinator
- /MotionArbiter

Protocolos

- AIP_ContractNet_MoverCaja
- AIP_MoveArbitration
- AIP_MetricsReporting

El diagrama del RobotAgent se enfoca en el comportamiento interno de un agente obrero cuya función principal es el transporte de cajas. En la sección de Atributos, se definen diversos elementos del estado interno del robot, incluyendo un identificador único, su ubicación dentro de un grid y varias variables que se relacionan con su tarea actual, tales como carrying, assignedCrateId, targetCell, targetStackCell y la ruta (path) asignada por el WarehouseManager. Además, el atributo status modela una máquina de estados simplificada que permite al robot transitar entre los estados Idle, ToPickup, ToDrop, Parked y Blocked. Es importante destacar que el diseño del robot no es completamente autónomo; en lugar de ello, sigue un esquema de coordinación centralizada donde las decisiones estratégicas son guiadas por el WarehouseManager y el robot se limita a ejecutar las tareas que le son asignadas.

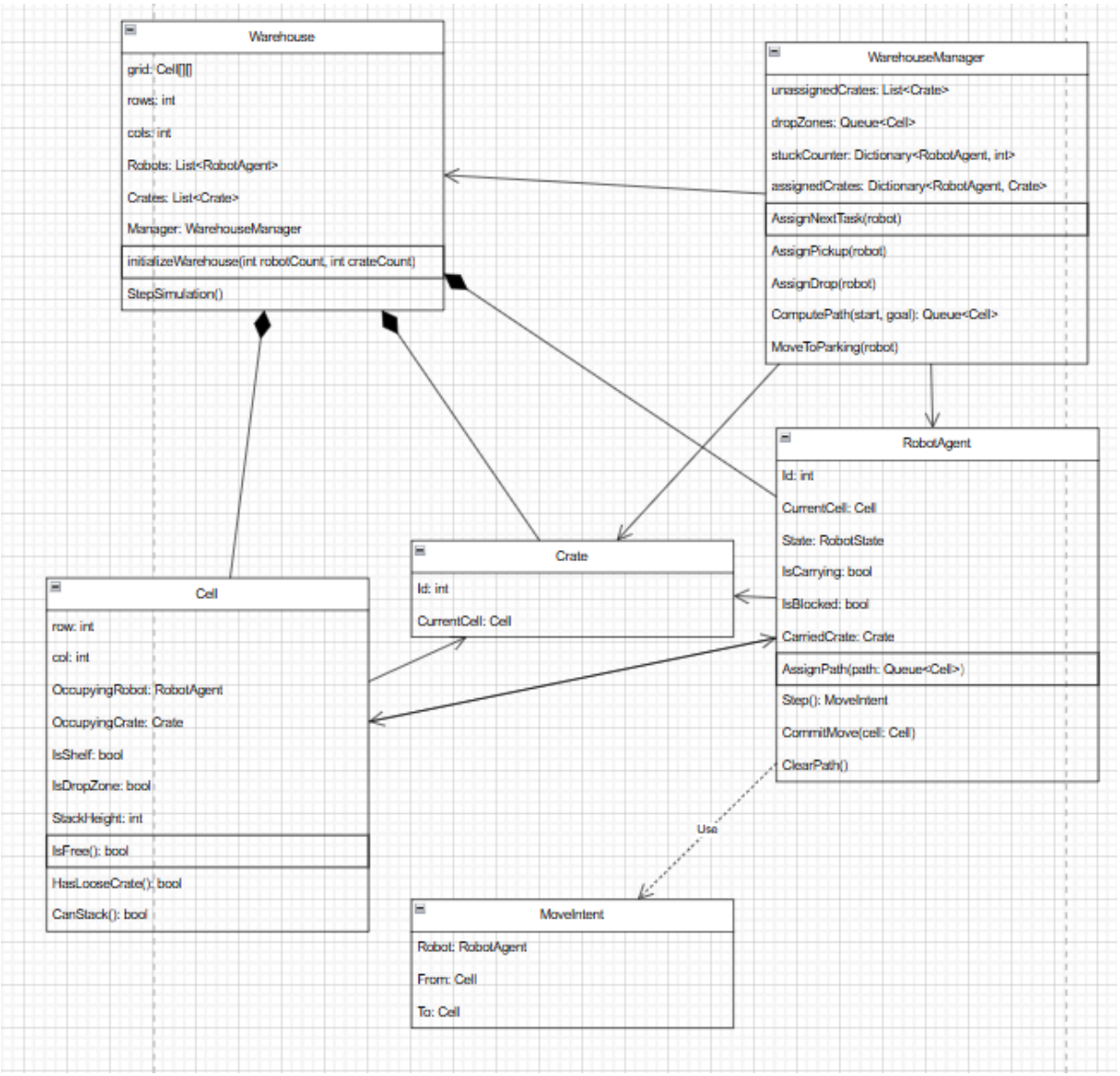
La sección de Percepción (Sensores) describe el componente reactivo del agente, el cual utiliza métodos como senseAdjacent(), senseCarrying(), senseNextCellFree() y senseCollisionRisk() para percibir su entorno inmediato. Esta capacidad de percepción es local y limitada, lo que se ajusta a su papel operativo en el sistema.

Por otro lado, la sección de Acciones aborda las capacidades físicas del robot mediante funciones como requestMove(), commitMove(), pickUp(), drop() y wait(). Esta solicitud permite una resolución centralizada de conflictos y la prevención de colisiones, asegurando un funcionamiento más eficiente. Además, el agente mantiene al Manager informado sobre eventos relevantes, tales como la finalización de tareas o bloqueos, lo que refleja una interacción sistemática entre el agente y el sistema de gestión. Esta estructura bidimensional de percepción y acción resalta la importancia de la comunicación y el control en la operativa del robot dentro de su entorno.
El apartado de Roles establece al robot como /Transporter y /Contractor, cuyo propósito principal es recoger y organizar cajas según las tareas asignadas.

El diagrama del WarehouseManager representa al coordinador del sistema, que mantiene una visión global del entorno (grid), el estado de los robots, información sobre las cajas y tareas pendientes, además de gestionar métricas como el número total de movimientos y el tiempo hasta completar la organización. En su componente de Percepción, el Manager recibe solicitudes de movimiento y eventos de los robots, actualizando el estado global del almacén para optimizar la toma de decisiones desde una perspectiva sistémica. En la sección de Acciones, el Manager publica tareas, asigna robots a cajas, calcula rutas, arbitra solicitudes de movimiento y autoriza desplazamientos, lo que implementa una estrategia de coordinación centralizada y control de tráfico interno.

Los Protocolos de Interacción formalizan la comunicación entre los agentes, destacando AIP_TaskAssignment para la asignación de tareas, AIP_MoveArbitration para la autorización de movimientos y AIP_TaskCompletion para la notificación de eventos.

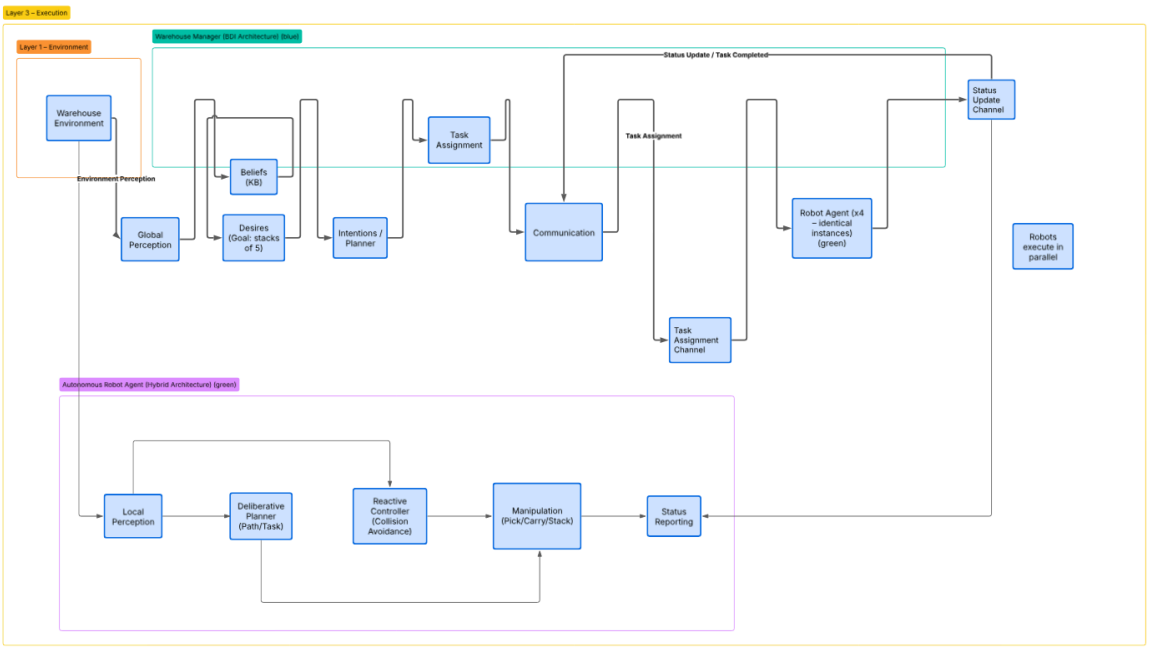
Diagrama UML



La clase Warehouse mantiene una relación de composición con Cell, RobotAgent y Crate, dado que es la encargada de crearlos e inicializarlos al comienzo de la simulación, estos objetos no existen de forma independiente al almacén. A su vez, Warehouse mantiene una asociación con WarehouseManager, ya que lo referencia para la coordinación de tareas sin ser responsable de su ciclo de vida. El WarehouseManager se asocia con múltiples instancias de RobotAgent (relación 1 a muchos) para asignarles tareas, calcular rutas y gestionar

situaciones de bloqueo. También se asocia con Crate para administrar la cola de cajas pendientes por recoger. RobotAgent mantiene una asociación con Cell a través de CurrentCell, representando su posición actual en la cuadrícula, y una asociación opcional (0..1) con Crate a través de CarriedCrate, indicando que un robot puede transportar como máximo una caja a la vez. La clase Cell referencia tanto a RobotAgent como a Crate para registrar qué objeto ocupa cada celda en un momento dado. Finalmente, RobotAgent genera instancias de MoveIntent en cada paso de simulación, lo que se representa como una dependencia, y utiliza el enumerador RobotState para modelar su estado interno, el cual puede ser Idle o Tasked.

Diagrama de interacción (AIP)



La figura muestra la arquitectura e interacción del sistema multiagente diseñado para la organización del almacén. El modelo se estructura en tres niveles principales: entorno, coordinación estratégica y ejecución distribuida.

En el Nivel 1 – Entorno, el Warehouse Environment representa el estado físico del almacén, incluyendo la ubicación de cajas, robots y posiciones de apilamiento. Este entorno no toma decisiones; únicamente provee información tanto al Warehouse Manager (percepción global) como a los robots (percepción local).

En el nivel de Coordinación Estratégica, el Warehouse Manager implementa una arquitectura deliberativa basada en el modelo BDI (Beliefs, Desires, Intentions). A partir de la percepción global del entorno, actualiza sus creencias sobre el estado del sistema, define el objetivo de formar pilas de hasta cinco cajas (deseos) y genera planes de acción mediante el módulo de intenciones. Posteriormente, asigna tareas específicas a los robots a través del canal de asignación (Task Assignment Channel) y recibe retroalimentación mediante el canal de actualización de estado (Status Update Channel), cerrando el ciclo de control.

En el Nivel 3 – Ejecución, los robots operan como agentes homogéneos que ejecutan tareas en paralelo. Cada robot implementa una arquitectura híbrida que combina un componente deliberativo, encargado de planificar rutas y ejecutar tareas asignadas, con un componente reactivo que prioriza la detección y evasión de colisiones.

Implementación Base de la Simulación (Parte 1)

Enfoque general del sistema

La implementación base de la simulación se desarrolló bajo un modelo de control centralizado total, en el cual el warehouse Manager actúa como entidad coordinadora global. Este agente posee visibilidad completa del estado del almacén en cada instante:

- Posición de todas las cajas.
- Posición y estado de cada robot
- Ubicación y estado de las pilas formadas
- Espacios libres y ocupados dentro de la cuadrícula M x N

El entorno lo modelamos como una matriz bidimensional discreta donde cada celda puede contener:

- Espacio vacío
- Pared
- Caja
- Robot
- Pila de cajas

Inicialización del Entorno

Al iniciar la simulación:

- Se generan K cajas en posiciones aleatorias válidas del almacén.
- Se colocan 5 robots en posiciones aleatorias vacías.
- Se generan estantes
- Todas las cajas comienzan a nivel de piso.
- Se define un tiempo máximo de ejecución.
- Se inicializan las estructuras de seguimiento:
- Contador global de movimientos.
- Tiempo total transcurrido.
- Registro de cajas organizadas.

Movimiento y Ejecución

Cada robot ejecuta su ruta en pasos discretos:

- Se mueve una celda por step.
- Incrementa el contador global de movimientos.

- Robots proponen a Warehouse un movimiento y actualizan su posición de ser aceptada
- Cuando el robot alcanza:
- Una caja: ejecuta acción pickUp() y cambia su estado a “Transportando”.
- El punto de depósito: ejecuta acción drop() y actualiza la pila correspondiente.

Formación de Pilas

Todas las cajas se transportan hacia una zona central del almacén destinada a la formación de pilas.

El Warehouse Manager:

- Lleva el control del número de cajas por pila.
- Asigna las posiciones de depósito.
- Verifica que ninguna pila exceda el límite de cinco cajas.

Resolución de Conflictos

Cuando dos robots intentan ocupar la misma celda o cruzar trayectorias:

- Se aplica una winner de manera aleatoria basada en ID.
- El otro robot es designado a no ir a ninguna parte hasta que la celda quede libre.

Este mecanismo:

- Previene bloqueos.
- Evita negociación compleja entre agentes.
- Mantiene simplicidad en el modelo centralizado.

Ventajas de la Estrategia Base

El enfoque centralizado ofrece:

- Eliminación de conflictos por asignación duplicada.
- Reducción de movimientos innecesarios.
- Optimización basada en proximidad.
- Comportamiento global predecible.
- Simplificación de la lógica individual de los robots.

Comparado con un enfoque distribuido puro (donde cada robot busca cajas de forma autónoma), esta estrategia reduce la probabilidad de que múltiples robots se dirijan al mismo objetivo y minimiza trayectorias redundantes.

Posibles Mejoras Futuras

A partir del análisis de resultados, se identifican posibles optimizaciones:

- División del almacén en zonas asignadas por robot.
- Reasignación dinámica si un robot queda bloqueado.
- Planificación con A* en lugar de trayectorias directas.
- Paralelización optimizada del depósito en múltiples puntos.
- Introducción de negociación distribuida entre robots.

Estas mejoras podrían disminuir aún más el tiempo total y el número de movimientos, acercando el modelo a sistemas logísticos inteligentes reales.

Estrategia Cooperativa y Optimización

La estrategia implementada en el sistema corresponde a un modelo de control centralizado total, donde el Warehouse Manager actúa como entidad coordinadora global. Este agente posee visibilidad completa del estado del almacén en todo momento, incluyendo la posición de todas las cajas, el estado y ubicación de cada robot, así como las posiciones designadas para la formación de pilas en el centro del entorno.

Asignación de Tareas

Cuando un robot se encuentra disponible, el Warehouse Manager ejecuta un proceso de asignación basado en proximidad. Para ello:

1. Calcula la distancia Manhattan entre el robot libre y cada caja disponible.
2. Selecciona la caja más cercana.
3. Reserva inmediatamente dicha caja para evitar asignaciones duplicadas.
4. Genera y envía al robot el camino completo hacia la caja y posteriormente hacia el punto de depósito.

En esta estrategia, el robot no realiza planificación autónoma, sino que ejecuta el camino recibido paso a paso. Esto garantiza coherencia global en la toma de decisiones y elimina comportamientos competitivos entre agentes.

Resolución de Conflictos

En caso de que dos robots coincidan en una intersección o ruta potencialmente conflictiva, Warehouse aplica una regla aleatoria de prioridad basada en el identificador del robot.

Este mecanismo simple evita bloqueos y reduce la complejidad de negociación entre agentes, manteniendo la consistencia del control centralizado.

Punto de Depósito y Formación de Pilas

Todas las cajas son transportadas hacia los 4 drop zones distribuidos en 4 cuadrantes del almacén, en una celda en específico.

Ventajas de la Estrategia

El enfoque centralizado presenta varias ventajas:

- Eliminación de conflictos por asignación duplicada.
- Reducción de movimientos innecesarios o en vacío.
- Optimización del tiempo total mediante asignación basada en proximidad.
- Coordinación coherente y predecible del sistema.
- Simplificación de la lógica de los robots individuales.

Comparado con una estrategia distribuida donde cada robot busca cajas de forma autónoma, este enfoque minimiza la probabilidad de que múltiples robots se dirijan al mismo objetivo o generen trayectorias redundantes. Como resultado, se reduce el número total de movimientos y el tiempo requerido para completar la organización del almacén.

Datos a Recopilar y Análisis

Tiempo y movimientos

Las métricas principales recopiladas durante la simulación son el tiempo total transcurrido hasta que todas las cajas quedaron organizadas en pilas de máximo cinco unidades, y el número acumulado de movimientos realizados por todos los robots. Estas métricas se monitorean en tiempo real a través de la interfaz HUD implementada en Unity.

Evolución de la estrategia

Durante el desarrollo se evaluaron dos estrategias de drop zone sucesivas. En la primera iteración, todas las cajas eran transportadas hacia un único punto central del almacén. Si bien esta estrategia simplificaba la lógica de asignación, generaba congestión en la zona central y aumentaba innecesariamente la distancia promedio recorrida por robots ubicados en los extremos del almacén.

En la segunda iteración se optó por dividir el almacén en cuatro cuadrantes, asignando una drop zone en el centro de cada uno. Esta distribución redujo significativamente la distancia

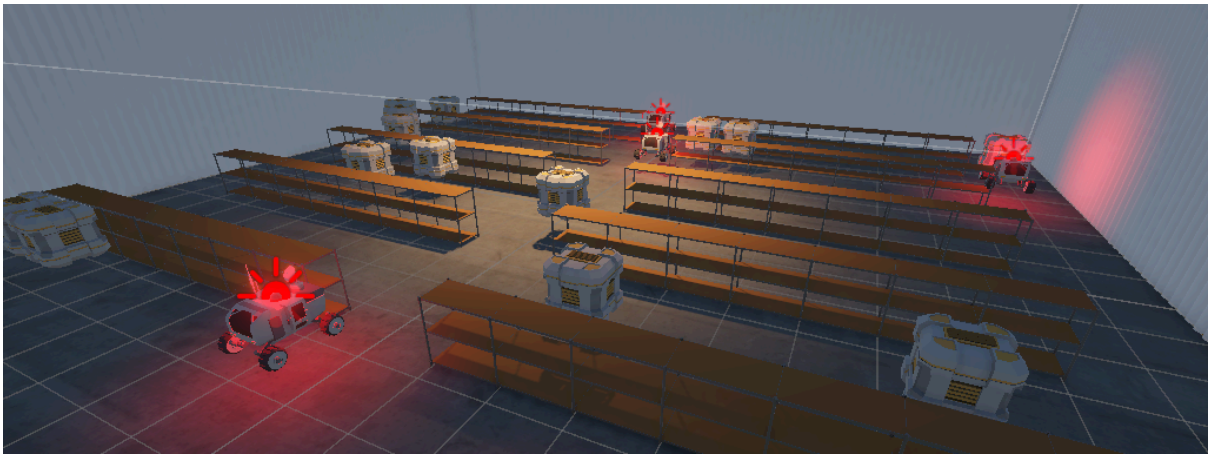
promedio de transporte y disminuyó los conflictos de tráfico en zonas compartidas, mejorando tanto el tiempo total como el número de movimientos.

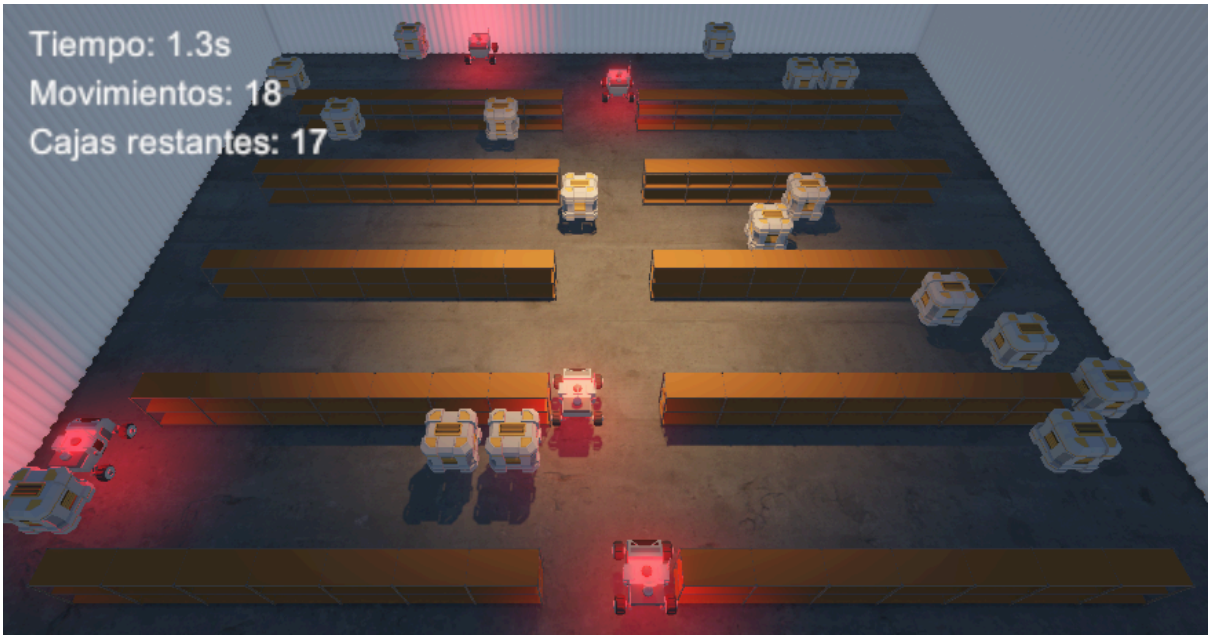
Estrategia optimizada propuesta

Una mejora adicional consistiría en calcular dinámicamente la posición de las drop zones a partir de la distribución real de las cajas generadas. Específicamente, se calcularía el centroide de la distribución sumando las coordenadas de todas las cajas y dividiendo entre el número total de ellas.

Este punto representa el centro de masa aproximado de la carga dentro del almacén. Posicionar las zonas de entrega en torno a este centroide, en lugar de en posiciones fijas predefinidas, permitiría adaptar dinámicamente la estrategia a cada configuración aleatoria del entorno. Como resultado, se minimizaría la distancia total recorrida por el conjunto de robots, reduciendo tanto el tiempo de ejecución como el número de movimientos necesarios para completar la organización del almacén.

Modelado 3D en Unity (Parte 2)





Detección de Colisiones e Integración Final

La detección y resolución de colisiones se implementó en dos niveles complementarios. A nivel lógico, en cada paso de simulación el método StepSimulation() recopila los MoveIntent de todos los robots y los pasa al módulo ResolveConflicts(), el cual identifica dos tipos de conflicto: colisiones directas, donde dos o más robots intentan moverse hacia la misma celda, y colisiones de intercambio, donde dos robots intentan cruzarse mutuamente. En

A nivel deliberativo, el WarehouseManager monitorea continuamente el estado de cada robot mediante el método Tick(). Si un robot permanece bloqueado durante un numero determinado de pasos consecutivos, el sistema intenta desbloquear la situación de dos formas: si el obstáculo es un robot inactivo, se le asigna una posición de estacionamiento libre; si el bloqueo persiste por otras causas, se cancela la tarea actual del robot y se le reasigna una nueva ruta recalculada desde su posición actual.
A nivel visual en Unity, los robots se desplazan de forma continua entre celdas mediante interpolación con Vector3.MoveTowards, sincronizando la posición del GameObject con el estado lógico en cada frame. Las cajas desaparecen visualmente al ser recogidas y se actualizan en altura al ser apiladas. Una interfaz en pantalla muestra en tiempo real el tiempo transcurrido, el número de movimientos realizados y las cajas restantes, permitiendo observar y analizar el comportamiento del sistema durante la ejecución.

Aprendizaje adquirido

A lo largo del desarrollo de esta actividad, como equipo logramos comprender de manera más profunda cómo funcionan los Sistemas Multiagentes cuando se enfrentan a un problema de coordinación en un entorno compartido. Aprendimos que no basta con que cada robot tenga la capacidad de moverse y recoger cajas; el verdadero reto está en diseñar reglas de comportamiento que permitan eficiencia colectiva, evitando bloqueos, colisiones y movimientos innecesarios.

Comprendimos la importancia de la percepción en los agentes: detectar paredes, pilas de cajas, robots y espacios libres fue clave para que las decisiones fueran coherentes con el entorno. También aprendimos a analizar métricas como el tiempo total de ejecución y el número de movimientos realizados, lo que nos permitió reflexionar sobre la eficiencia de nuestra estrategia inicial y considerar mejoras, como asignación de zonas, reducción de trayectorias redundantes o priorización de cajas cercanas.

En la parte de Gráficas Computacionales 3D, adquirimos experiencia práctica en la construcción de un entorno tridimensional funcional. Aprendimos a trabajar con prefabs reutilizables para estantes, cajas y robots, aplicar materiales y texturas mediante mapeo UV, e implementar iluminación adecuada para mejorar la visualización del almacén. También fortalecimos nuestro entendimiento sobre animación de movimiento y detección básica de colisiones, integrando la lógica del sistema multiagente con su representación visual.

De manera integral, el aprendizaje más significativo fue entender cómo conectar la lógica algorítmica con la simulación visual. No solo desarrollamos un sistema que resuelve el problema, sino que logramos visualizarlo y analizar su comportamiento en tiempo real. Además, fortalecimos nuestras habilidades de trabajo en equipo, división de responsabilidades y organización del proyecto, comprendiendo que la coordinación entre nosotros es tan importante como la coordinación entre los robots.